

BIND-VLA: Boundary-Gated Agentic Supervision and Quality-Aware Inference

Abstract—Frozen vision-language-action (VLA) policies can generate action chunks, but long-horizon manipulation also depends on remembering task context and deciding when a local action warrants correction. Adding an agentic supervisor creates a second problem: a premature override can interrupt useful motion, while repeated semantic inference can make the system slower. We present BIND-VLA, a harness that separates high-rate execution monitoring from lower-rate semantic decisions and admits ordinary subgoal changes only at action-chunk boundaries. Task-scoped memory supplies source-traceable context; an inference-profile gate considers replay validity, resource cost, and closed-loop quality before deployment. In an exploratory eight-task RoboMME study, success changes from 23/80 to 42/80 across development variants. A fixed identity-memory comparison on VideoUnmaskSwap changes 5/12 to 9/12, but transfer to VideoUnmask changes 15/16 to 13/16, and a frozen two-task Raw/Harness comparison changes 3/32 to 4/32. Separately, a PyTorch $\pi_{0.5}$ profile reduces model-call P95 latency from 282.43 to 54.67 ms on an RTX 4090. These results motivate controlled intervention and cost-aware deployment, while leaving general closed-loop gains and integrated acceleration unestablished.

Index Terms—Vision-language-action models, embodied agents, robot manipulation, memory, efficient inference

I. INTRODUCTION

Vision-language-action (VLA) models connect language and perception to robot control, from OpenVLA to the $\pi_{0.5}$ family [1], [2]. Their action chunks are local, however: a long task may require remembering a demonstrated target after occlusion, deciding whether a grasp or placement actually completed, and selecting the next skill. RoboMME makes such temporal and identity-dependent demands explicit [3]. Harness VLA suggests wrapping a frozen policy in higher-level capabilities [4], but an external agent is not automatically helpful. Its decisions can be wrong, and its multimodal calls add latency even when the underlying policy is unchanged.

One failure in our development runs illustrates the control problem. A monitor treated gripper closure as evidence that a grasp had succeeded and switched the instruction to placement before the action was complete. The error was not in the VLA weights: a low-cost process signal had been granted authority over task semantics. Conversely, invoking a VLM after every action chunk can spend more time planning than acting. Faster model calls alone do not resolve this tradeoff; realtime VLA systems likewise distinguish network latency from continuous robot execution [5].

We therefore ask: *what evidence should authorize an agent to change a frozen policy’s next subgoal, and at what runtime cost?* BIND-VLA separates a high-rate monitor, which

reports execution signals, from a lower-rate vision-language model (VLM), which interprets task context. Task-scoped memory carries provenance rather than issuing actions. An authority gate checks semantic proposals and applies ordinary subgoal changes only at an action-chunk boundary; an independent safety interlock, when available, can still halt unsafe motion immediately. The frozen VLA remains the low-level controller. A separate profile gate screens policy inference configurations against latency, memory, replay validity, and task quality. Figure 1 shows how these decisions surround, rather than replace, the action policy.

Our contribution is a controlled way to compose agentic supervision with a frozen action policy. We define a two-rate authority contract in which execution signals can request review but cannot certify task completion or rewrite an active chunk. We implement source-traceable memory and boundary receipts, then measure both rescued and harmed paired episodes rather than counting only successful recoveries. Finally, we couple the harness to a quality-aware inference-profile gate and report model-call speed separately from closed-loop cost. The evaluated evidence concerns one VLA family; it does not establish online policy learning, hard realtime control, or cross-policy generalization.

II. RELATED WORK

Planning, failure recovery, and memory. VLM-based correction and replanning are established in robotic manipulation: ReplanVLM handles errors during task planning and execution [6], while anomaly-triggered pause, perturbation, and reset have been tested on physical robots [7]. A hierarchical VLM/LLM planner with learned action masks also addresses subtask switching without an extra termination-model call [8]. Harness VLA exposes frozen VLA behavior as retryable capabilities [4], and AHA uses a VLM to reason about manipulation failures [9]. We do not claim to invent semantic replanning, detection, or recovery. BIND-VLA instead specifies when external evidence may authorize the *next* action chunk of an unchanged policy, and audits rescues alongside harmful interventions. MemoryVLA embeds memory in the policy [10]; our task-scoped records are external and source-traceable. Long-horizon benchmarks such as CALVIN evaluate composed language-conditioned behavior [11]; RoboMME provides the temporal and identity-dependent tasks used here to test both memory benefit and negative transfer [3].

Efficient execution and evaluation. TinyVLA improves speed through a compact policy architecture [12]. PointVLA keeps an action expert frozen but learns a lightweight 3D

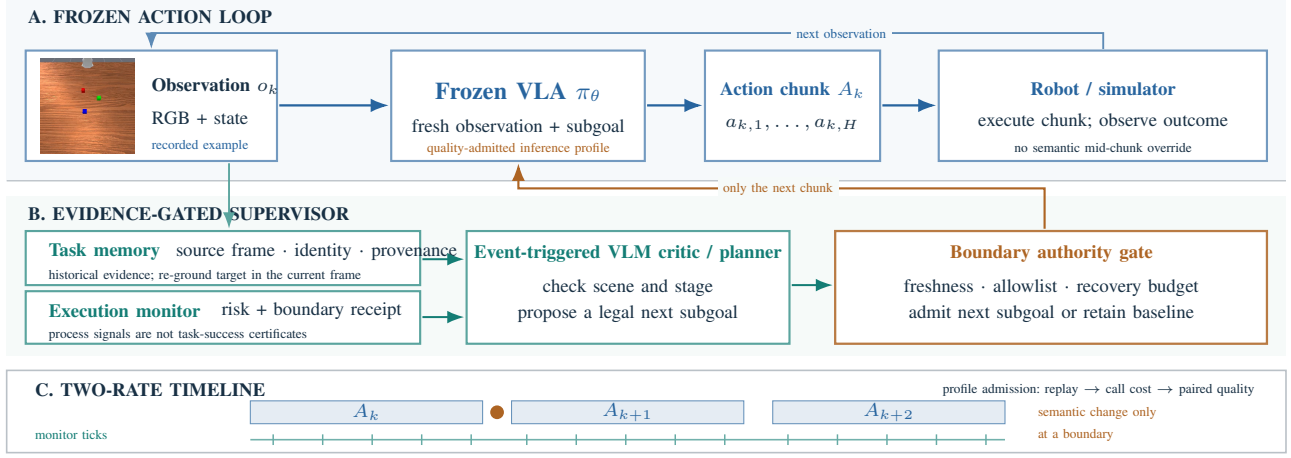


Fig. 1. BIND-VLA places checked semantic supervision around a frozen VLA action loop. A monitor reports process evidence; task memory and an event-triggered VLM supply context, but ordinary subgoal changes take effect only at a chunk boundary. A screened inference profile configures the policy call. The thumbnail is a recorded RoboMME demonstration frame; the evaluated GroundSG-skill adapter is narrower than the general interface.

feature injection module [13]; BIND-VLA instead leaves the policy weights and representations unchanged. This preserves a plug-in control interface but cannot itself repair missing visual features. Realtime VLA systems address policy inference and action continuity [5], [14], [15]. RILaaS measures inference serving under robot deployment loads [16], illustrating why model-call latency and full execution cost must be reported separately. Our profile study combines existing backend settings with a paired-quality admission rule, rather than proposing compilation, quantization, or a new VLA backbone. LIBERO-PRO probes robustness beyond memorized LIBERO trajectories [17]; our local ten-state subset is not its official leaderboard protocol.

III. AGENTIC EXECUTION AND INFERENCE CONTROL

A. Execution authority at action boundaries

At chunk boundary k , let o_k be deployable visual/proprioceptive observations, ℓ the task instruction, m_k a bounded memory record, and b_k the remaining call/recovery budget. The frozen policy proposes a chunk

$$A_k = \pi_\theta(o_k, \ell_k) = (a_{k,1}, \dots, a_{k,H}). \quad (1)$$

The controller executes the admitted portion, collects a receipt e_k from the robot/simulator adapter, and updates its state only from observations and receipts. Evaluator success flags, online oracle subgoals, and privileged simulator object poses are excluded from decisions. The monitor can flag stalling, action age, deadline failures, or inconsistent progress. Such a flag is a *request for review*, not proof of grasp or subtask completion.

In the reusable runtime, an escalated VLM decision has one typed intent from $\{\text{continue}, \text{vla_act}, \text{run_skill}, \text{safe_stop}\}$. The gate checks schema, freshness, capability allowlist, and budget. Memory may supply object identity and provenance but cannot issue joint commands; the VLM may select a textual subgoal or registered skill, not torques or trajectories.

The RoboMME evaluation uses a narrower adapter: its GroundSG planner returns task-specific language subgoals, which are checked against the legal skill grammar and fail closed on invalid output. Thus the paired RoboMME results test boundary receipts and memory on one PI0.5/VLM chain, not every typed tool of the general runtime. The policy-adapter interface is replaceable, but cross-policy performance is untested.

B. Memory, stage state, and non-interruption

The memory record is indexed by task and episode and carries source observation, target identity, and provenance. Admission checks its task scope and source; the VLM must re-ground a historical target in the live frame. An ordered stage state separates local motion from task completion: gripper closure is an execution signal, not proof of a correct grasp. The controller never rewrites a running action chunk on this signal alone. At the next boundary, it can present an execution receipt to the VLM and request a legal next skill, retry within budget, or stop. This corrects an earlier monitor/planner authority inversion without asserting that a receipt alone verifies successful placement.

For the VideoUnmask task family, an offline compiler uses only the public initial demonstration. It detects color-coded cubes before occlusion and their nearest white containers at the first covered frame, rejecting associations beyond 20 pixels. SAM 2 [18] tracks each associated container to the end of the video; excessive mask loss rejects the record. The retained color-to-container map, requested order, and source-video hash are passed to the VLM as *historical evidence*, not as verified current positions or completion flags. The VLM must still ground the next target in the live frame. Compilation time is accounted for separately from rollout latency.

C. Selective semantic calls and profile admission

Let r_k denote monitor risk and v_k indicate whether a cached subgoal remains valid under the new observation. A

VLM call is requested when an event is raised, the cached subgoal is invalid, or its reuse budget expires. Otherwise the current subgoal is reused and the policy sees a fresh o_k ; *only the semantic plan is cached, not the action chunk*. In the current implementation, validation and fallback occur at chunk boundaries rather than by speculative mid-chunk commands. This distinction matters because an apparently faster scheduler can reduce episode duration simply by failing earlier.

Writing E_k for an event raised from r_k or a task-state change and c_k for the remaining reuse count, the semantic-call proposal is

$$q_k = \mathbb{I}\{E_k \vee \neg v_k \vee c_k = 0\}. \quad (2)$$

Regardless of q_k , a new low-level chunk is generated from the current observation. If $q_k = 1$, the returned semantic intent must pass the authority gate before it can change the *next* subgoal; it never rewrites a chunk already executing.

For deployment accounting, we distinguish policy computation from the rest of the control loop. Per-episode wall time comprises observation handling, VLA calls, semantic VLM calls, synchronization, and environment or robot execution. This is an accounting identity, not five independently measured timings in every experiment. Selective VLM calls can reduce one term while increasing policy calls or prolonging a trajectory; quality-admitted profiles must therefore be checked against *complete* paired episodes, not only single-call P95.

An inference profile identifies checkpoint, backend, precision, flow-step count, view mask, action horizon, and target hardware. Admission has three gates: matched-output replay, warm single-call latency and memory under a declared budget, and paired closed-loop task quality. A profile that meets a latency target but worsens task quality is rejected or retained only for a separately labeled low-memory tier. The reported 80-ms threshold below is a *model-call target*, not a guarantee on sensing, planning, and robot actuation together. In particular, a candidate is not deployed merely because its measured T_{95} is lower: it must also meet the predeclared paired-quality rule $Q(P) \geq Q(P_0) - \epsilon$ and the memory budget. In our conservative screens, an observed harm can reject a candidate even when the mean success difference is small; this sample gate is not a population-level non-inferiority test.

IV. EXPERIMENTAL PROTOCOL

We separate three non-interchangeable experimental settings. The historical RoboMME study uses the official GroundSG PI0.5 checkpoint 79999 and a Qwen3-VL-4B GroundSG planner, 16-action chunks, and a 1300-step limit. Eight of RoboMME’s 16 tasks are covered with 10 fixed episodes per task and condition. B1 invokes the planner every chunk without harness memory/tools; C2 adds memory, grounded tools, and procedure/temporal control while retaining every-chunk planning; C3 adds selective planning and validated subgoal reuse. The same task/episode identities

are paired. All success values are official environment outcomes. This is a *historical development-and-expansion study*: C3 aggregates several task-specific development revisions, and the last four tasks were added after the first four had been studied. The nominal paired statistics below therefore describe the recorded sample; they are not a pre-registered confirmation of one frozen final implementation.

To isolate identity memory on the same action-policy and VLM chain, a later VideoUnmaskSwap gate preselected episodes 30–37, followed by a separately preselected confirmation range 38–53. The official test split has only 50 episodes: four out-of-range IDs 50–53 returned duplicate preflight states and were excluded *before any model rollout*, without replacement. The confirmation therefore has 12 valid, previously unused paired states, 38–49. A retains the full public initial demonstration but no structured memory; B adds only color-to-container identity memory compiled from that same demonstration. Both arms use the same frozen models, 16-step chunks, camera, planner-at-every-chunk schedule, 1300-step budget, and model-service start; order alternates A–B and B–A by episode parity. Stage-receipt and identity-conflict corrections are disabled. Rejected memories and all official failures, including budget-exhausted ongoing runs, are retained.

For task outcomes, we report both total successes and paired transitions: a *rescue* means the baseline failed and the compared method succeeded on the same task/episode, whereas a *regression* reverses that outcome. A recovery event within one rollout is not automatically a rescue. Exact McNemar tests use only discordant paired outcomes. Planner calls, policy calls, and elapsed wall time are recorded separately, since reducing one count does not imply an end-to-end speedup. We distinguish all-episode totals from the cost of *both-successful* pairs; the latter avoids calling an early failure “fast.”

The separate LIBERO experiment uses a PyTorch PI0.5 LIBERO checkpoint and Qwen3.5-4B semantic planner. Four locally available ten-task suites (standard LIBERO-10 and three LIBERO-PRO variants) are evaluated on 10 initial states per task, 400 paired states per condition. This is not the official 50-initial-state leaderboard. Finally, the model-call experiment uses the same PyTorch PI0.5 family on an RTX 4090 and 45 paired recorded observations with fixed noise and action horizon 10. Its numbers must not be transferred to the RoboMME JAX implementation.

V. RESULTS AND EVIDENCE BOUNDARIES

A. Exploratory long-horizon study

Table I contains the complete historical eight-task aggregate. Relative to B1, C2 has 18 failure-to-success and 5 success-to-failure transitions; C3 has 22 and 3. The corresponding nominal exact McNemar p values are 0.0106 and 0.000157. Because C3 includes development variants, these values should not be interpreted as confirmatory significance for the final harness. Relative to C2, C3 saves 813 planner calls (40.6%) and 1455.2 seconds of cumulative wall time (28.1%) but makes more policy calls (2169 versus 2001); its

TABLE I

HISTORICAL ROBOMME STUDY: EIGHT TASKS, 80 PAIRED EPISODES PER ARM. THESE ARE EXPLORATORY DEVELOPMENT/EXPANSION RESULTS, NOT ONE FROZEN CONFIRMATION RUN.

Condition	Success	Planner	Policy	Wall (s)
B1 Raw	23/80	2141	2141	5208.0
C2 Harness	36/80	2001	2001	5178.7
C3 + selective	42/80	1188	2169	3723.6

TABLE II

TASK-LEVEL ROBOMME SUCCESSSES (OF 10 EACH). HETEROGENEITY MATTERS: THE STRONGEST GAINS ARE NOT UNIFORM ACROSS TASKS.

Task	B1	C2	C3
StopCube	0	3	2
VideoRepick	2	3	7
RouteStick	1	3	5
VideoUnmaskSwap	2	7	8
BinFill	7	8	8
ButtonUnmask	0	3	2
PickHighlight	3	2	2
MoveCube	8	7	8

six-success advantage has nominal $p = 0.146$. Cost and task quality must be read together.

The historical B1-to-C2 transition has five harms alongside 18 rescues. C2-to-C3, which isolates scheduling more closely, has nine rescues and three harms; its nominal test does not establish that the scheduler improves success. C3 also makes 168 more low-level policy calls than C2. Thus fewer planner calls do not imply faster policy inference or a uniformly better controller.

The four later-added tasks yield 18/40, 20/40, and 20/40 for B1/C2/C3. Earlier Swap development screens varied across service starts; neither they nor this aggregate establish a frozen multi-task effect.

B. Fixed identity-memory gate and within-task confirmation

Table III isolates public-demonstration identity memory rather than the full multi-module harness. All eight gate and twelve valid confirmation memories pass the admission check; every A/B pair has matching initial-observation hashes and runs within one service start. Confirmation yields five rescues and one harm, a descriptive 33.3-point success difference but exact two-sided McNemar $p = 0.21875$. Three rescues occur in six single-target tasks; the six two-target tasks have two rescues and one harm. This is directional within-task evidence, not statistical significance or cross-task generalization.

Figure 3 shows Swap confirmation ep39. After the first placement, B grounds the red-hidden second container near (98, 94) and succeeds; A repeatedly targets around (81, 112) and fails. This is consistent with identity memory changing the semantic target. The paired Swap counterexample ep43 follows the correct green-to-blue order in both arms, yet B fails and A succeeds. Identity correctness does not guarantee the physical action trajectory. Moreover, rescued Swap ep47 requires 1075 steps and 68 VLM/policy calls with B, so a

TABLE III

ROBOMME IDENTITY-MEMORY A/B. A SEES THE PUBLIC DEMONSTRATION; B ADDITIONALLY RECEIVES TRACKED IDENTITY MEMORY. OFFICIAL SUCCESS; BUDGET EXHAUSTION COUNTS AS NON-SUCCESS. UNMASK IS A SEPARATE TRANSFER TEST.

Set	A success	B success	Rescues	Harms
Swap gate ep30–37	2/8	4/8	2	0
Swap confirm ep38–49	5/12	9/12	5	1
Unmask ep22–37	15/16	13/16	1	3



Fig. 2. Direction of outcome changes in fixed RoboMME pairs. Left bars count baseline successes lost; right bars count baseline failures rescued. Memory rows isolate demonstration-derived identity; harness rows compare frozen Raw and Harness configurations. The tasks were selected for targeted tests, not as an unbiased RoboMME average.

higher completion count cannot be read as lower deployment cost. On only four both-successful confirmation pairs, A/B use 46/43 planner calls and 60.47/57.91 seconds; these differing trajectories provide no evidence for the separate Optimize Runtime’s effect.

The fixed second-task transfer test uses 16 previously unused VideoUnmask episodes and the same public-video SAM2 compiler. Unlike an earlier pilot in which a motion gate suppressed all memory hints, B now actually receives the admitted identity record in every episode. A/B succeed in 15/16 and 13/16 respectively (one rescue, three harms; exact two-sided $p = 0.625$). All twelve single-target pairs succeed in both arms, while success on four two-target pairs changes from 3/4 to 1/4. In two harms, B repeatedly attempts the first pickup; another exhausts its 1300-step budget in a placement loop. Thus a correctly sourced identity record does not ensure robust stage execution. The twelve both-successful pairs use 134 versus 100 planner/policy calls and 152.74 versus 121.41 seconds, but differing trajectories and the omitted SAM2 build cost (21.67 seconds for these pairs, excluding shared model load) preclude an unconditional efficiency claim. Always-on transfer is rejected. An additional matched Unmask development pilot (ep38–45) yields 8/8, 8/8, and 7/8 for Raw, observation-only Harness, and Harness+memory: no rescue and one harm. We then added an opt-in boundary receipt recording that a put-down chunk ran and naming the next demonstration target; it neither certifies placement nor interrupts that chunk. The pilot’s failed ep43 succeeds on rerun (365 steps), a targeted repair rather than an independent rescue. On unseen two-target ep47, frozen Raw, Harness, and Harness+memory+receipt all succeed (313, 313, and 327 steps). This tests the receipt path but shows no success-rate or efficiency gain.

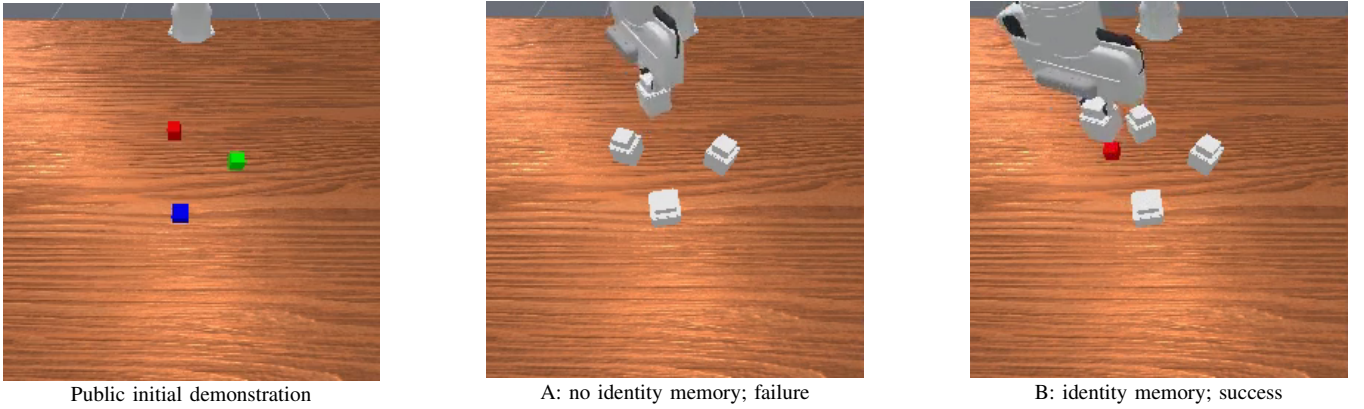


Fig. 3. Qualitative paired VideoUnmaskSwap episode 39. The left frame is from the public demonstration; the other frames are the terminal top-view rollout images, not synchronized counterfactual states. A’s final VLM target is (80, 112) and it fails; B’s is (98, 93) and it succeeds. This illustrates how demonstration-derived identity can change a later target, but is one case, not a population-level effect estimate.

A same-chain B/C ablation asks whether selective VLM scheduling can mitigate this overhead without losing task quality. Using the same memory and models, B plans every chunk and C uses validated subgoal reuse. On eleven strict same-service pairs, B/C succeed in 8/11 and 7/11, with one rescue and two harms. In the six both-successful pairs, planner calls fall from 45 to 29 and total wall time from 66.84 to 52.13 seconds, while policy calls change from 45 to 44. Yet the predeclared quality gate rejects C because of the two harms. In the twelfth episode, C proposes an unsupported action skill and is safely rejected at 112 steps; B is completed in a second service start and succeeds. This cross-service case is excluded from strict paired quality and timing totals, and the C error is not relabeled as an official task failure. These observations demonstrate an overhead–quality tradeoff, not an admitted Optimize profile.

C. Frozen Raw/Harness task stress test

To assess whether the current implementation reproduces the exploratory multi-task trend, we fix two task families selected for opposing historical behavior: RouteStick previously improved, while PickHighlight had a regression. On 16 new paired episode identities per task, both arms use the same GroundSG PI0.5, Qwen3-VL, observations, and every-chunk schedule; Harness alone adds its existing task-specific planning contract, without identity memory or the rejected selective schedule. RouteStick yields Raw/Harness success of 0/16 versus 1/16; PickHighlight yields 3/16 versus 3/16. Combined, the frozen comparison is 3/32 versus 4/32, one rescue, zero harms, exact two-sided McNemar $p = 1$. Three pairs succeed under both arms; their combined wall time is 53.43 versus 55.01 seconds and planner/ policy calls 58 versus 60. Two Harness runs exhaust the control budget, compared with one Raw run. The task families were purposefully chosen from historical positive and regression cases, not randomly sampled from RoboMME; their poor absolute success also limits how much high-level supervision can be assessed. The fixed test therefore bounds the exploratory result: it does not show a strong Harness gain or whole-system acceleration.

TABLE IV

LIBERO LOCAL TEN-STATE SUBSET BY SUITE; EACH ROW IS 10 TASKS $\times$ 10 STATES. NOT OFFICIAL 50-STATE LEADERBOARD SCORES.

Suite	Frozen	Fixed	Agentic
LIBERO-10	96/100	97/100	97/100
Object perturbation	66/100	67/100	68/100
Swap perturbation	11/100	11/100	11/100
Task perturbation	7/100	6/100	7/100

D. LIBERO robustness and agent overhead

On 400 locally paired states, frozen PI0.5 succeeds 180 times, fixed recovery 181, and the agentic variant 183. The frozen-to-agentic difference is 0.75 percentage points (four rescues, one regression; exact McNemar $p = 0.375$). VLA calls fall from 16047 to 14699 (8.4%), but cumulative wall time rises from 7610.7 to 8864.6 seconds (16.5%) as 263 semantic-planning calls are added. The clean LIBERO-10 subset is already 96/100 for the frozen policy, whereas the more perturbed Swap and Task subsets remain 11/100 and 7/100 for both frozen and agentic conditions (Table IV). This experiment supports a concrete systems motivation—the semantic layer can dominate total cost—but not a broad success or end-to-end speed claim.

E. Independent inference-profile measurements

Figure 4 reports single-call P95 on 45 paired recorded observations. The 7-step eager and 2-step compiled profiles differ in both flow-step count and backend, so their ratio is not attributable to one optimization. Within the 2-step configurations, compilation and static masked-view elision (SMVE) further lower measured latency. The current late-language INT8 route saves memory but is too slow and is rejected. Fidelity checks on 45 records for V2–V4 concern the defined replay comparison; they do not prove that 7-step and 2-step policies have equal closed-loop quality.

For Qwen3.5-4B, NF4 reduces allocated model memory from 8.46 to 3.08 GiB while increasing P95 from 1896.55 to 2909.79 ms. Thirty recorded semantic cases agree exactly; this is a memory tier, not a latency optimization. One

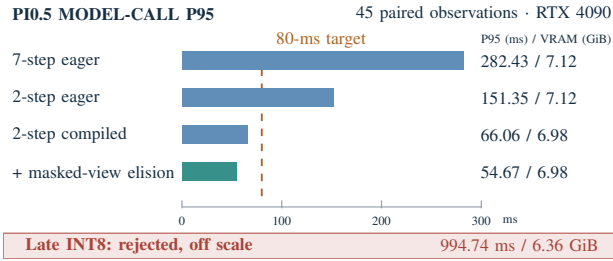


Fig. 4. Independent PyTorch PI0.5 model-call profiles on RTX 4090 (45 paired observations). The dashed line is an 80-ms *call* target; INT8 is separately labeled because its 994.74-ms P95 exceeds the plotted range. Different step counts cannot be credited to compilation alone. These are not end-to-end JAX RoboMME timings.

co-resident NF4-planner/SMVE-policy smoke test occupies 11.22 GiB but withholds robot actions; it demonstrates model integration only.

There is a limited closed-loop profile gate on two saved recovery snapshots (tasks 06 and 09) from the PyTorch LIBERO line. Compile+SMVE BF16 succeeds on both; late-language INT8 alone also succeeds on both but is slower and thus only a lower-memory option. The composed INT8+SMVE profile fails one of the two and is rejected. These branches test a fixed physical-recovery continuation, *not* the VLM-guided Agentic Harness, and two snapshots do not estimate population-level task success. They do, however, show why a profile should be admitted by quality as well as by isolated latency.

VI. DISCUSSION AND LIMITATIONS

The central systems variable is *intervention authority*. Gripper closure and other inexpensive signals can flag risk without proving that a subtask is complete. A source-correct identity mapping can still harm a rollout if stage execution is wrong: Swap memory rescues five confirmation pairs but loses one, and always-on Unmask transfer loses three. The monitor can therefore run frequently while semantic changes wait for a fresh image, a VLM decision, and the next chunk boundary. The repaired Unmask ep43 shows that this path is executable; unseen ep47 shows that it need not be faster.

These observations do not establish a statistically robust gain for a frozen integrated system. The eight-task aggregate mixes development revisions, the identity-memory confirmation is non-significant ($p = 0.21875$), and a retrospective motion check remains untested in closed loop. Selective planning fails its paired quality gate even where both-successful trajectories use fewer calls. Initial-state matching does not prevent later trajectories from diverging. The PyTorch PI0.5 profile is a separate model-call result, not acceleration of the JAX RoboMME robot chain. We test one action-policy family, without a physical robot. In contrast, recent RA-L recovery and hierarchical-execution studies include physical-robot or cross-platform tests [7], [8]; our simulator-only evidence does not meet that validation standard. A submission-strength integrated claim still needs a frozen multi-task comparison and a quality-admitted, same-chain Optimize

profile.

VII. CONCLUSION

BIND-VLA treats agentic intervention as an authority-and-budget decision around a frozen VLA. Boundary-timed supervision prevents inexpensive process signals from directly rewriting an active action chunk. In paired tests, source-traceable memory helps one identity-dependent task but harms blind transfer, and the frozen Raw/Harness comparison shows only a small change. Separately, quality-screened profiles reduce PI0.5 model-call latency, not the measured JAX robot-chain time. The resulting design supports conditional, auditable supervision; its joint task-quality and efficiency benefit remains to be established in a frozen multi-task test.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, *et al.*, “OpenVLA: An open-source vision-language-action model,” 2024. arXiv:2406.09246.
- [2] Physical Intelligence, K. Black, N. Brown, *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” 2025. arXiv:2504.16054.
- [3] Y. Dai, H. Fu, J. Lee, *et al.*, “RoboMME: Benchmarking and understanding memory for robotic generalist policies,” 2026. arXiv:2603.04639.
- [4] Y. Zhang, H. Zhang, F. Gao, *et al.*, “Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents,” 2026. arXiv:2607.08448.
- [5] Y. Ma, Y. Zhou, Y. Yang, T. Wang, and H. Fan, “Running VLAs at real-time speed,” 2025. arXiv:2510.26742.
- [6] A. Mei, G.-N. Zhu, H. Zhang, and Z. Gan, “ReplanVLM: Replanning robotic tasks with visual language models,” *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 10201–10208, 2024.
- [7] B. Santhanam, A. Mitrevski, S. Thoduka, S. Houben, and T. Hassan, “Reliable robotic task execution in the face of anomalies,” *IEEE Robotics and Automation Letters*, vol. 11, no. 1, pp. 314–321, 2026.
- [8] N. Gu, Y. Li, K. Kosuge, and M. Hayashibe, “Language-driven multi-task manipulation with action-mask-enhanced multimodal learning,” *IEEE Robotics and Automation Letters*, vol. 11, no. 5, pp. 5550–5557, 2026.
- [9] J. Duan, W. Pumacay, N. Kumar, *et al.*, “AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation,” 2024. arXiv:2410.00371.
- [10] H. Shi, B. Xie, Y. Liu, *et al.*, “MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation,” 2025. arXiv:2508.19236.
- [11] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [12] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, F. Feng, and J. Tang, “TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation,” *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3988–3995, 2025.
- [13] C. Li, J. Wen, Y. Peng, Y. Peng, and Y. Zhu, “PointVLA: Injecting the 3d world into vision-language-action models,” *IEEE Robotics and Automation Letters*, vol. 11, no. 3, pp. 2506–2513, 2026.
- [14] C. Yang, Y. Hu, Y. Ma, *et al.*, “Realtime-VLA v2: Learning to run VLAs fast, smooth, and accurate,” 2026. arXiv:2603.26360.
- [15] J. Niu, K. Gu, Y. Zhao, *et al.*, “Realtime-VLA FLASH: Speculative inference framework for diffusion-based VLAs,” 2026. arXiv:2605.13778.
- [16] A. K. Tanwani, R. Anand, J. E. Gonzalez, and K. Goldberg, “RILaaS: Robot inference and learning as a service,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4423–4430, 2020.
- [17] X. Zhou, Y. Xu, G. Tie, *et al.*, “LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization,” 2025. arXiv:2510.03827.
- [18] N. Ravi, V. Gabeur, Y.-T. Hu, *et al.*, “SAM 2: Segment anything in images and videos,” 2024. arXiv:2408.00714.